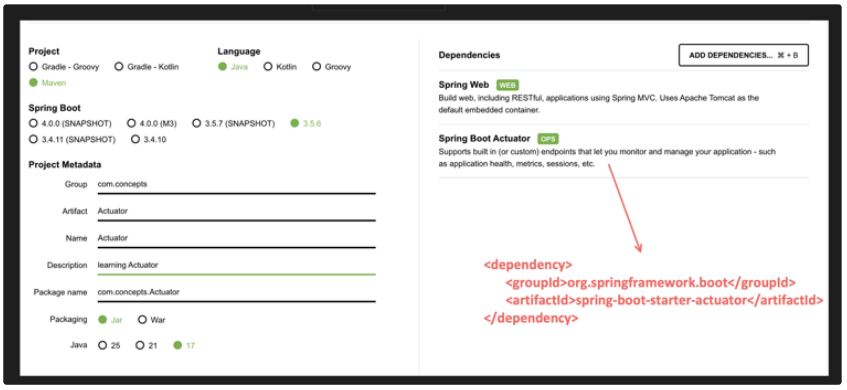
Springboot Actuator

Actuator:

Provides production-ready endpoints to monitor and manage the Spring boot application.

Project setup:

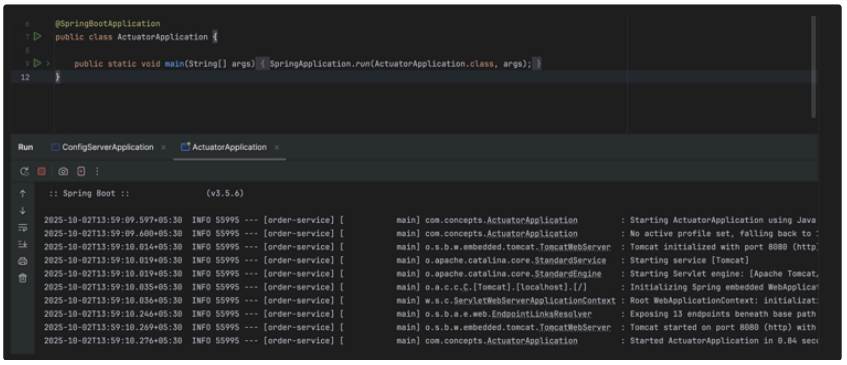


application.properties

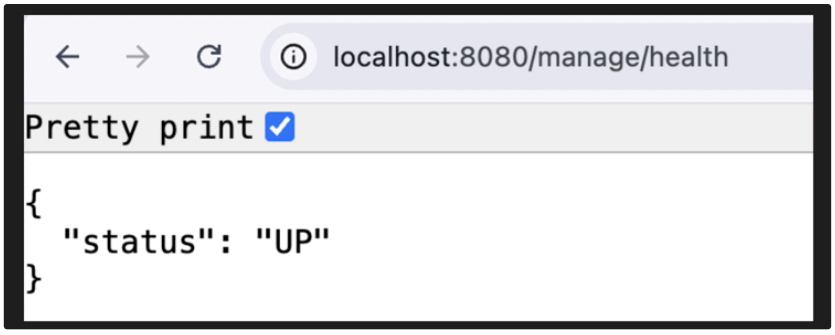
```
spring.application.name=order-service

# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=*

```

Concept && Coding

Let's start the server and hit the Actuator endpoints:



Health API : '/health'



Metrics API : '/metrics'

'/metrics' gives list of all metrics endpoint available

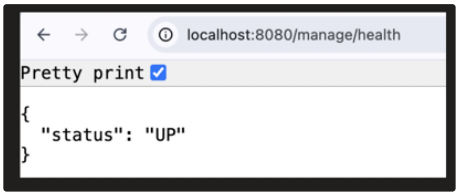
Concept && Coding

Hitting specific metrics endpoint

Lets first see some useful in-built Actuator endpoints:

GET: /health

Provides health status of our application: UP, DOWN, OUT_OF_SERVICE, UNKNOWN



/health endpoint can be extended to add additional checks like DB health, Cache status etc. beyond just application server status

Adding DB health check:

```
@Component
public class DatabaseHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        boolean isDBUp = checkDBConnection();
        return isDBUp ? Health.up().withDetail("DB", "Available").build()
                      : Health.down().withDetail("DB", "Not-Available").build();
    }

    private boolean checkDBConnection() {
        //check  DB is up or not
        return true;
    }
}

```

Concept && Coding

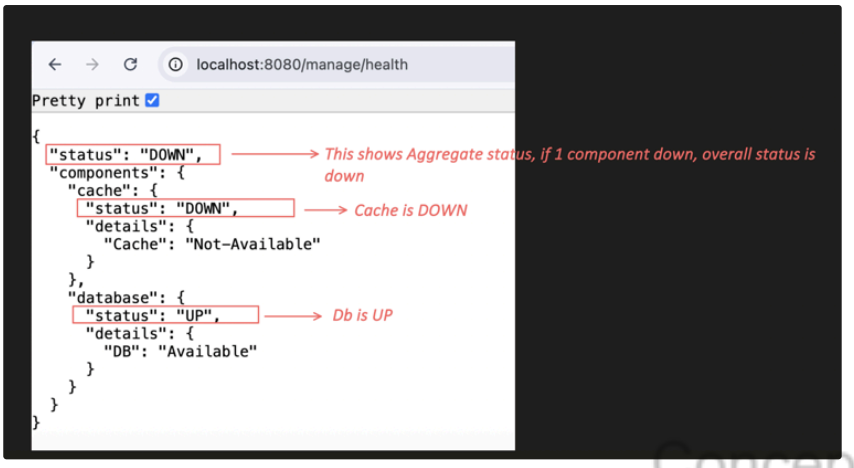
Adding Cache health check:

```
@Component
public class CacheHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        boolean isCacheUp = checkCacheStatus();
        return isCacheUp ? Health.up().withDetail("Cache", "Available").build()
                         : Health.down().withDetail("Cache", "Not-Available").build();
    }

    private boolean checkCacheStatus() {
        //check  cache status
        return false;
    }
}

```

By-default, its shows only the overall status of the application

But if we want more details, we need to add:

```
spring.application.name=order-service

# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=*

#to view the details in health endpoint
management.endpoint.health.show-details=always

```



Concept && Coding

GET: /metrics

and

/metrics/{metric name}

There are so many metrics, just showing some important ones:

	Metric name	What it represents	Example
JVM Memory Metrics			
	jvm.memory.used	Memory currently in use by JVM	{ "statistic": "VALUE", "value": 99478616 }
	jvm.memory.max	Max memory JVM can use	{ "statistic": "VALUE", "value": 10989076477 }
Garbage Collection Metrics			

	jvm.gc.pause	Time spent in GC.
		COUNT: total no. of GC events occurred.
		TOTAL_TIME: total time spent in GC (usually seconds)
		MAX: longest single GC pause observed
			[
			{
			"statistic": "COUNT",
			"value": 12
			},
			{
			"statistic": "TOTAL_TIME",
			"value": 2.305
			},
			{
			"statistic": "MAX",
			"value": 0.9
			}
			]

Threads			

	jvm.threads.live	Number of live threads	{
			"statistic": "VALUE",
			"value": 22
			}

	jvm.threads.peak	Peak live thread count, since JVM started	{
			"statistic": "VALUE",
			"value": 50
			}

System Metrics			
	system.cpu.usage	CPU used by JVM (range 0.0 - 1.0)	Value: 0.10 → 10%
HTTP Server / Requests			
	http.server.requests	COUNT: Total no of HTTP requests received TOTAL_TIME: total time spent handling all requests (usually in seconds) MAX: longest time taken to handle a single request	"measurements": [{ "statistic": "COUNT", "value": 152 }, { "statistic": "TOTAL_TIME", "value": 23.45 }, { "statistic": "MAX", "value": 0.89 }], "availableTags": [{ "tag": "method", "values": ["GET", "POST"] },

Concept && Coding

			{ "tag": "status", "values": ["200", "404", "500"] }] }
Database/ JDBC Metric			
	jdbc.connections.active	Connections currently in use	VALUE: 3
	jdbc.connections.idle	Idle connections in pool	VALUE: 7
	jdbc.connections.max	Max connections allowed	VALUE: 10

Concept && Coding

GET: /threaddump

- **Helps to diagnose deadlock or thread leaks.**
- **Which threads are active, blocked or waiting.**
- **Stack trace for each thread (what code it is executing)**
- **Thread name, id, priority etc.**


```
[
 {
   "threadName": "main",
   "threadId": 1,
   "blockedTime": -1,
   "blockedCount": 0,
   "waitedTime": -1,
   "waitedCount": 0,
   "threadState": "RUNNABLE",
```

```
    "stackTrace": [
      "com.concepts.MyService.methodName(MyService.java:142)",
      "com.concepts.ActuatorApp.main(ActuatorApp.java:10)"
    ]
  },
  {
    "threadName": "thread2",
    "threadId": 2,
    "blockedTime": -2,
    "blockedCount": 0,
    "waitedTime": -1,
    "waitedCount": 0,
    "threadState": "WAITING",
    "stackTrace": [
      "com.concepts.ClassName.methodName(ClassName.java:12)",
      "com.concepts.ActuatorApp.main(ActuatorApp.java:10)"
    ]
  }
]
```


Concept && Coding

What all other API's available:
Official Documentation: Endpoints :: Spring Boot

/heapdump	GET	Downloads JVM heap as .hprof file
/mappings	GET	Lists all Spring MVC request mappings
/beans	GET	Displays a complete list of all the Spring beans in your application
/configprops	GET	Lists all @ConfigurationProperties beans
/loggers	GET	Lists all loggers and their current levels
/shutdown	POST	Lets the application be gracefully shutdown
/env	GET	Shows environment properties
/actuator/env/{property}	GET	Shows a specific environment property

By default, access to all endpoints except for "/shutdown" and "/heapdump" is unrestricted

Both these endpoints are very critical:

- /shutdown can stop your application
- /heapdump can expose sensitive information about your application like tokens, password etc.

  So, by default they are restricted and specifically want us to unrestricted it (assuming, we have accepted the risk involved)

```
# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=*

#to view the details in health endpoint
management.endpoint.health.show-details=always

# Make it require authentication
management.endpoint.heapdump.access=unrestricted

```

<u>Security:</u>

We have already covered Spring Security in depth (total 9 parts), so kindly check it out, if there is any doubt with that:

Concept && Coding

pom.xml

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

```

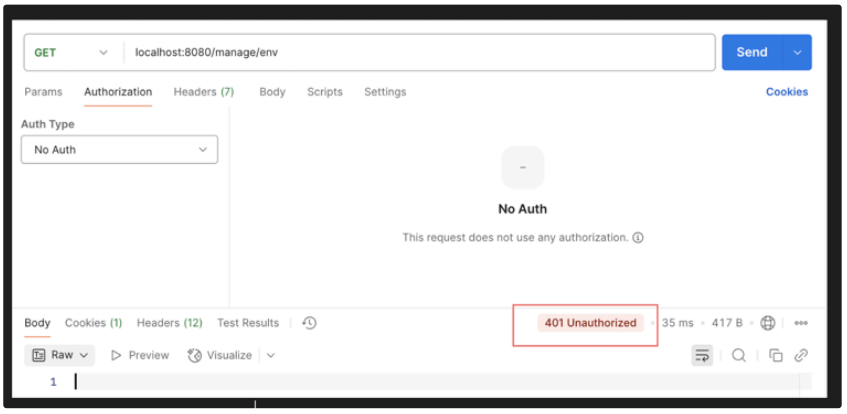
```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{

        http.authorizeHttpRequests(auth -> auth
                        .requestMatchers("/manage/health", "/manage/info").permitAll() // public
                        .anyRequest().authenticated() // everything else requires login
```

```
                )
                .httpBasic(Customizer.withDefaults()); // basic auth for simplicity
        return http.build();
    }
}

```

application.properties

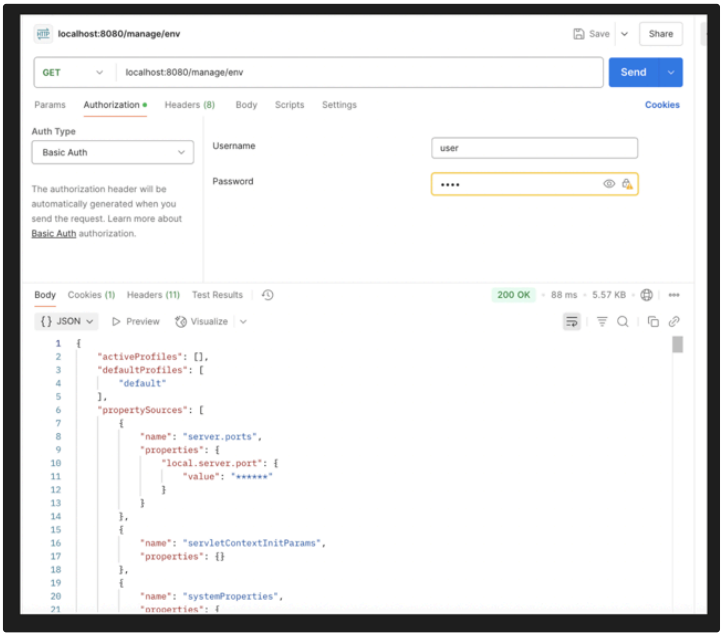
```
spring.security.user.name=user
spring.security.user.password=pass
spring.security.user.roles=ADMIN

```

Trying to access /evn without providing the authentication, request denied:



Concept && Coding

Able to access, after successful Authentication:



<u>Custom Actuator Endpoint:</u>

Class annotated with @Endpoint(id = "custom endpoint name")

id → forms the **URL path**: /actuator/{id}

```
@Component
```

```
@Endpoint(id = "my-custom-stats")
public class MyCustomStatsEndpoint { ... }

```

// our endpoint becomes '/actuator/my-custom-stats'

Type of operation, our custom endpoint going to support:

1. **@ReadOperation**

(equivalent to HTTP GET

2. **@WriteOperation**

(equivalent to HTTP POST)

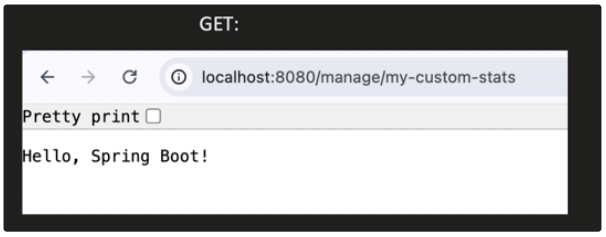
3. **@DeleteOperation**

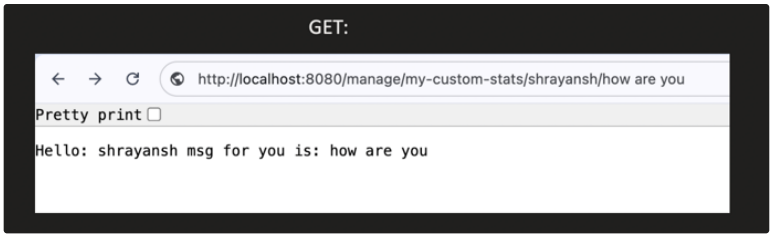
(equivalent to HTTP DELETE)

```
@Component
@Endpoint(id = "my-custom-stats")
public class MyCustomStatsEndpoint {

    @ReadOperation
  /*
  Return type can be any serializable object like Map,
  List, String, POJO, primitive etc.
  */
    public String readAll() {
        return "Hello, Spring Boot!";
    }

   @ReadOperation
  /*
  /my-custom-stats/{name}/{message}
  selector follows sequence
  */
    public String read(@Selector String name, @Selector String message) {
        return "Hello: " + name + " msg for you is: " + message;
    }
}

```

Concept && Coding

```
spring.application.name=order-service

# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=my-custom-stats,health,info

```






```java
@Component
@Endpoint(id = "my-custom-stats")
public class MyCustomStatsEndpoint {

    @WriteOperation
    public String refresh() {
          // simulate say cache refresh
           return "refreshed";
    }

   @DeleteOperation
    public String remove(@Selector String key) {
        return "reset done for key: " + key;
    }
}

```

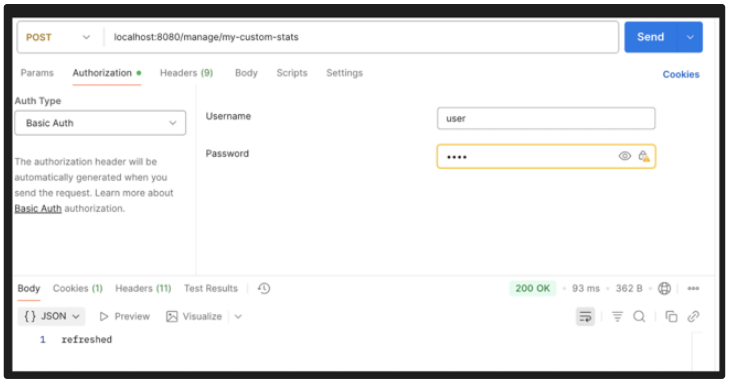
For POST and DELETE operation, it requires authentication

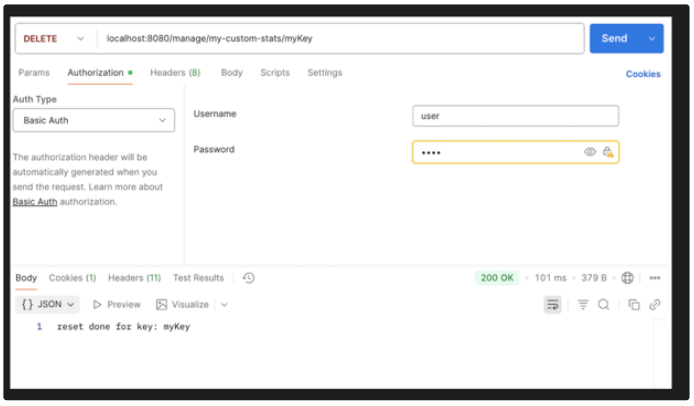
```java
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception{

        http.authorizeHttpRequests(auth -> auth
                        .anyRequest().authenticated())
                .csrf(csrf -> csrf.disable())
                .httpBasic(Customizer.withDefaults()); // basic auth for simplicity
        return http.build();
    }
}

```

Concept && Coding

application.properties

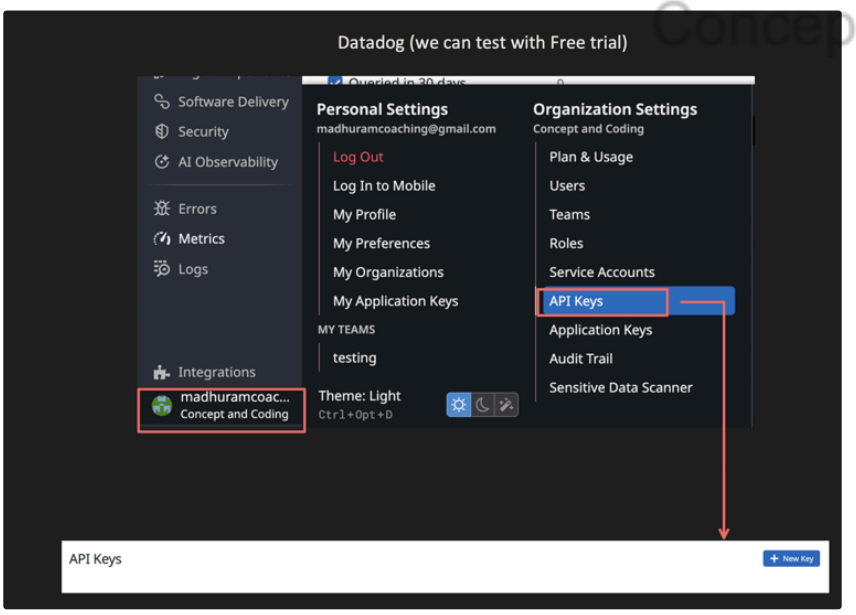
```properties
spring.application.name=order-service

# by-default path is /actuator
management.endpoints.web.base-path=/manage

# Expose all actuator endpoints, by-default '/actuator/health' & '/actuator/info' endpoint is exposed
# '*' expose all the endpoints
# use comma separated like: health, info, metrics, loggers etc. to expose selected endpoints
management.endpoints.web.exposure.include=my-custom-stats,health,info

spring.security.user.name=user
spring.security.user.password=pass
spring.security.user.roles=ADMIN

```





Pushing the metrics to Datadog (Monitoring platform)

we can also push to different other platforms like: Prometheus, CloudWatch etc..



Concept && Coding

application.properties

```
management.datadog.metrics.export.apiKey=e59af51cb6ae3a24129da7230e5dd9f7
<!--
When its true then only it try to push the metrics
-->
management.datadog.metrics.export.enabled=true
  <!--
  Every 5s it will push the metrics to datadog
  -->
management.datadog.metrics.export.step=5s

```

pom.xml

```
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-datadog</artifactId>
</dependency>

```

All the metrics of Spring Boot Actuator are automatically pushed to **Datadog**.

Concept && Coding